EEP 524
Applied High-Performance GPU Computing

LECTURE 2 : 13 October, 2026

Instructor: Dr. Colin Reinhardt

ECE Professional Masters Program

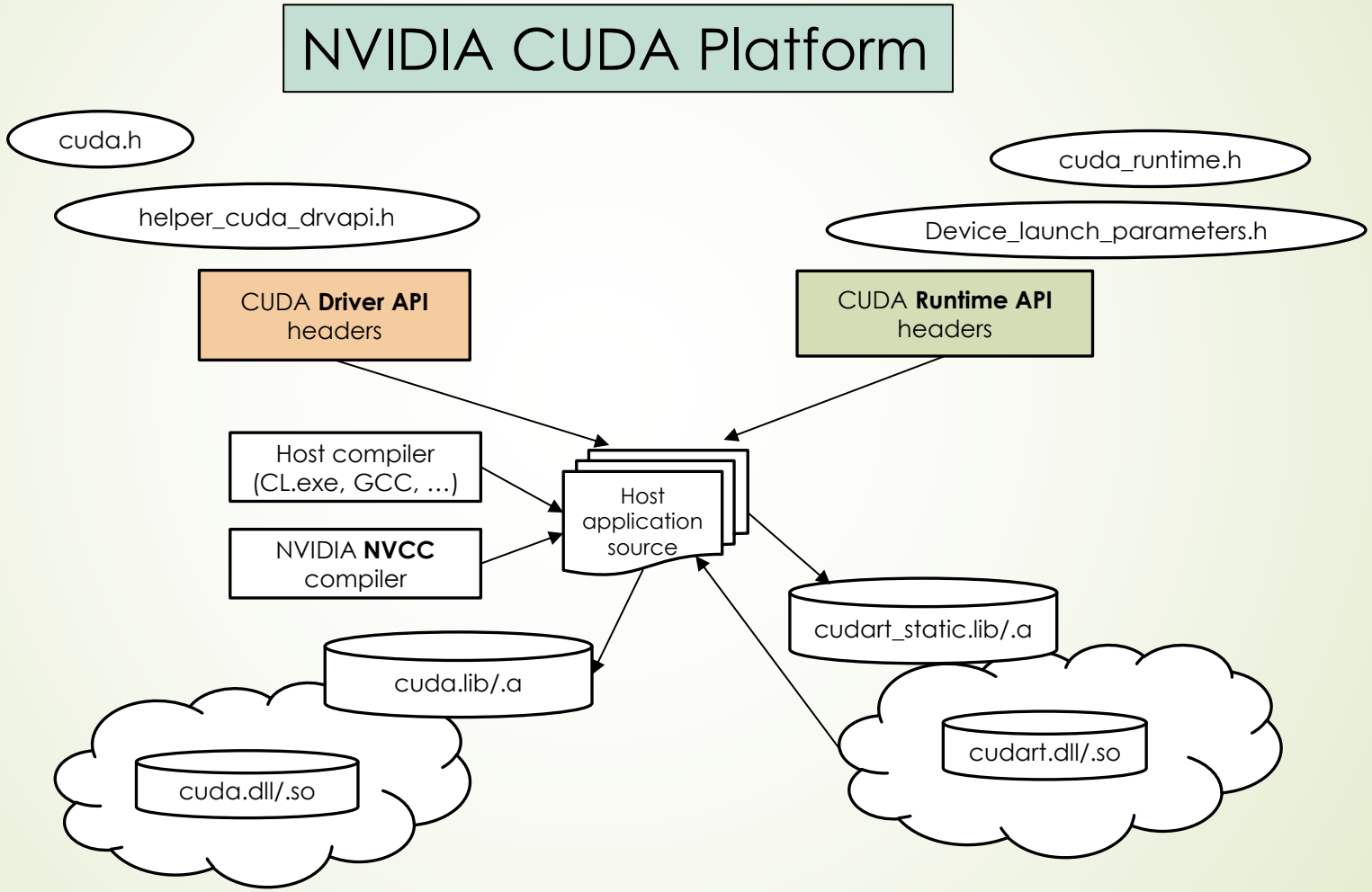
Autumn 2026

Lecture 2 : Outline

- NVIDIA CUDA Platform and CUDA Compilation
- Fundamental CUDA concepts and techniques
  - data dimensionality and memory representation
  - computational domain (index space) dimensionality
  - mapping problem & index spaces to HW execution
- CUDA Device kernel language
- GPU Hardware Architecture: NVIDIA Turing T4
- Introduction to Nsight VSE kernel debugger

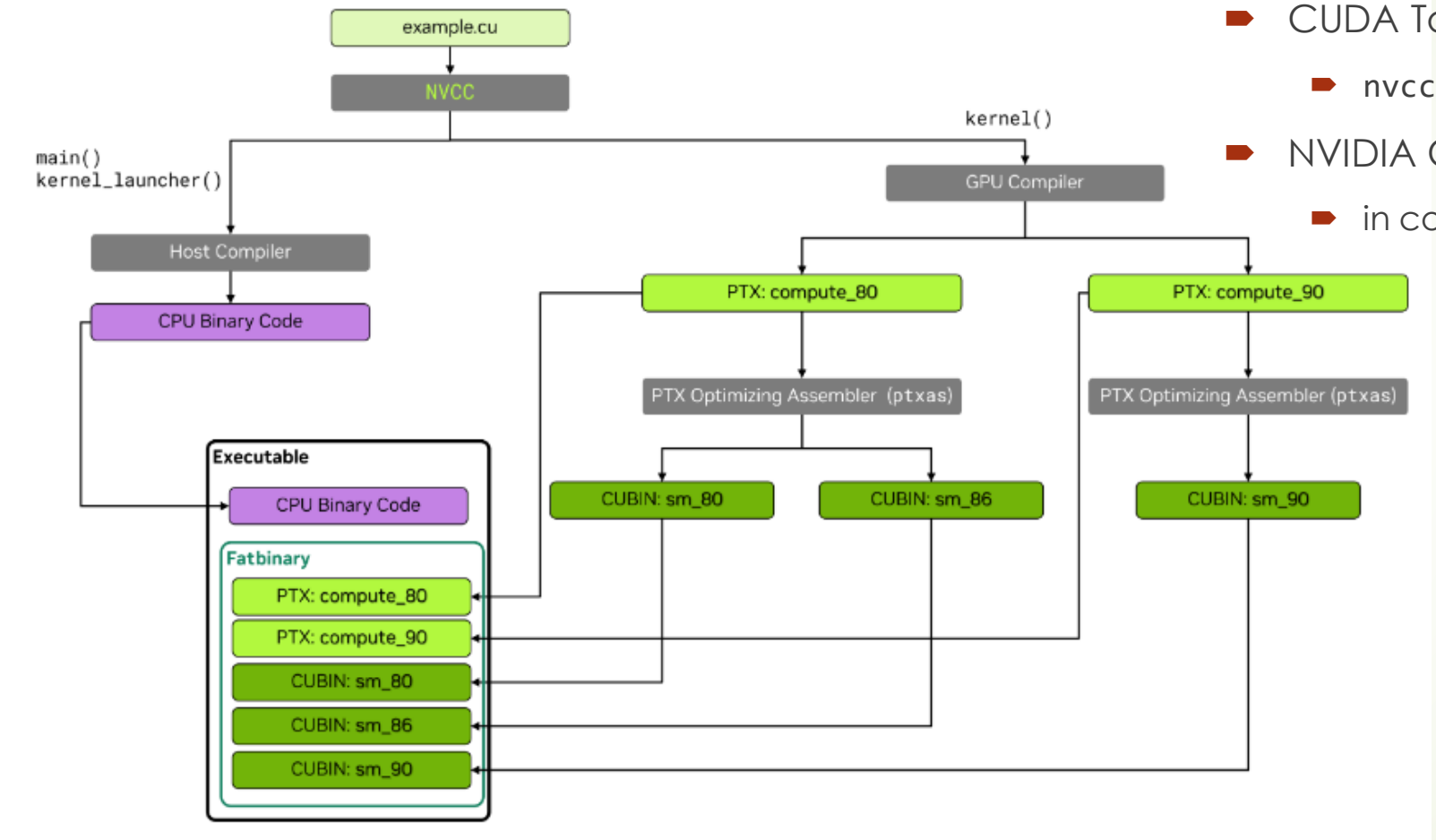
- HW1
  - DUE 20 OCT (TUE), 5:59 PM
    - submission will be online via class web Assignments page

GPU Compute Environment Components



All CUDA support from NVIDIA CUDA Toolkit

CUDA Compilation: Phases & Trajectory



- CUDA Toolchain
  - `nvcc`, linker, PTX and Cubin assemblers
- NVIDIA CUDA Compiler nvcc :offline compilation
  - in contrast to JIT compilation by `nvrtc`

- NVCC basic usage

```
nvcc example.cu -I path_to_include/ -L
path_to_library/ -lcublas -o
<output_file>
```

PTX: virtual GPU architecture(s)

- `compute_XY`

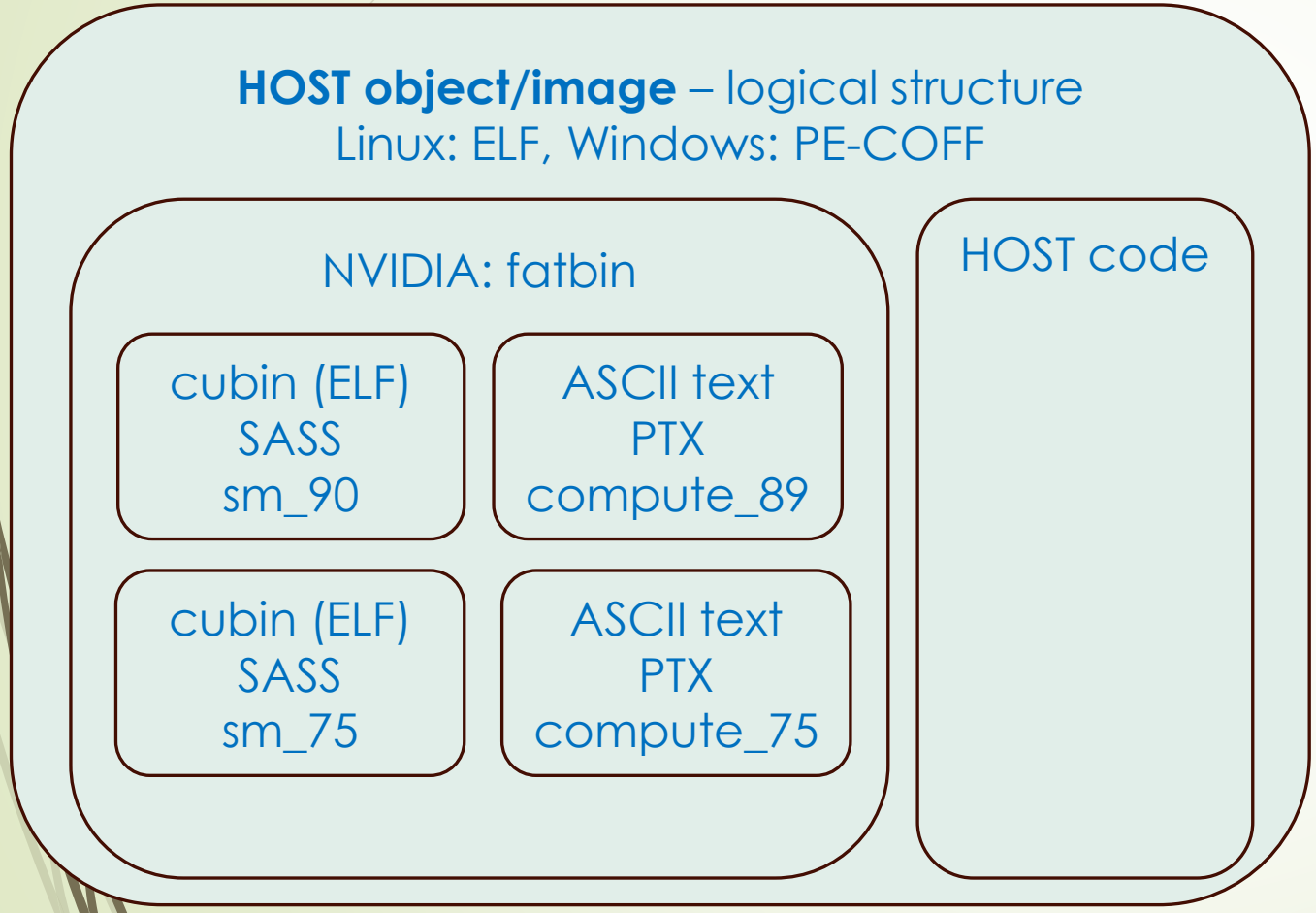
Cubin (SASS): real GPU architecture

- `sm_XY`

```
# generate PTX for virtual arch compute_80 and compile it to Cubin for real arch sm_86, keep compute_80 PTX
nvcc example.cu -arch=compute_80 -gpu-code=sm_86,compute_80 # (PTX and Cubin)

# (1) generate PTX for virtual architecture compute_80 and compile it to Cubin for real architecture sm_86, sm_89
nvcc example.cu -gencode=arch=compute_80,code=[compute_80,sm_86,sm_89]
```

HOST / GPU binary executable format



Inspecting the binary on Windows

Host binary .exe

```
dumpbin /headers vecadd.exe
```

NVIDIA CUDA artifacts: PTX, SASS

```
cuobjdump -lelf vecadd.exe
cuobjdump -lptx vecadd.exe
```

DEMO

Intro to

- AWS WorkSpaces Applications VDE
- Visual Studio 2026 IDE Interface:
  - Project Properties
- CUDA Toolkit 13.x
  - Compilation trajectory with **`nvcc`**

Data Dimensionality: application view and computer view

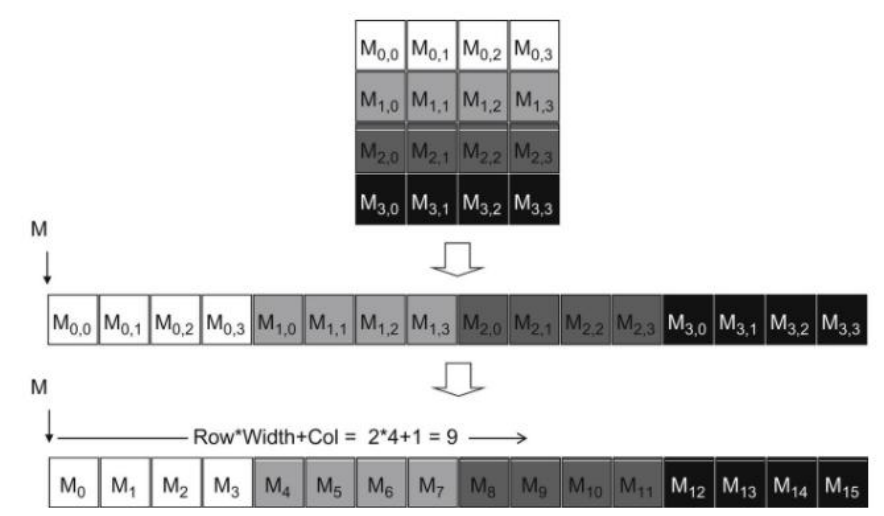
- An application domain expert may think of the data for a problem in its "natural" dimensionality
  - image processing: 2D h(x,y): e.g. images
  - physical problem: 3D f(x,y,z) : e.g. physical volumetric space
- However, computer memory space is a "flat" address space.
  - the compiler and processor reference memory using a base + offset
  - 1D linear, contiguous
- Thus, all higher-dimensional data representations are ultimately "linearized"
  - "flattened" into equivalent 1D arrays
  - multi-dimensional indexing syntax translated into 1D offsets
  - consider: c-style multi-dimensional arrays
    - x[i][j]
    - y[i][j][k]
    - * note left-most index is "slowest moving", right-most is "fastest"

Recall: Little-endian representation

- When you look at multi-byte values (float32, int32, double) in memory on an Intel x86 / x64 architecture the values are stored in **little-endian** form
- **Byte Order:** The bytes are ordered from the smallest part of the number to the largest part.
- **Example:** Take the 4-byte hexadecimal number `0x12345678` (where `0x12` is the most significant byte and `0x78` is the least significant byte).
- **In Memory:** A little-endian system stores the bytes in reverse order across sequential memory addresses:
  - Address `0x00`: `0x78`
  - Address `0x01`: `0x56`
  - Address `0x02`: `0x34`
  - Address `0x03`: `0x12`

"flattened" (linearized) arrays

- Consider 2D matrix (2D C array) : M(row,col) = M(y,x) = $M_{y,x}$ = m[y][x]
- There are two basic choices to linearize into a 1D representation
  - row-major : concatenate successive rows
  - column-major : concatenate successive columns
- Standard C, CUDA, and OpenCL all adopt the **row-major** convention

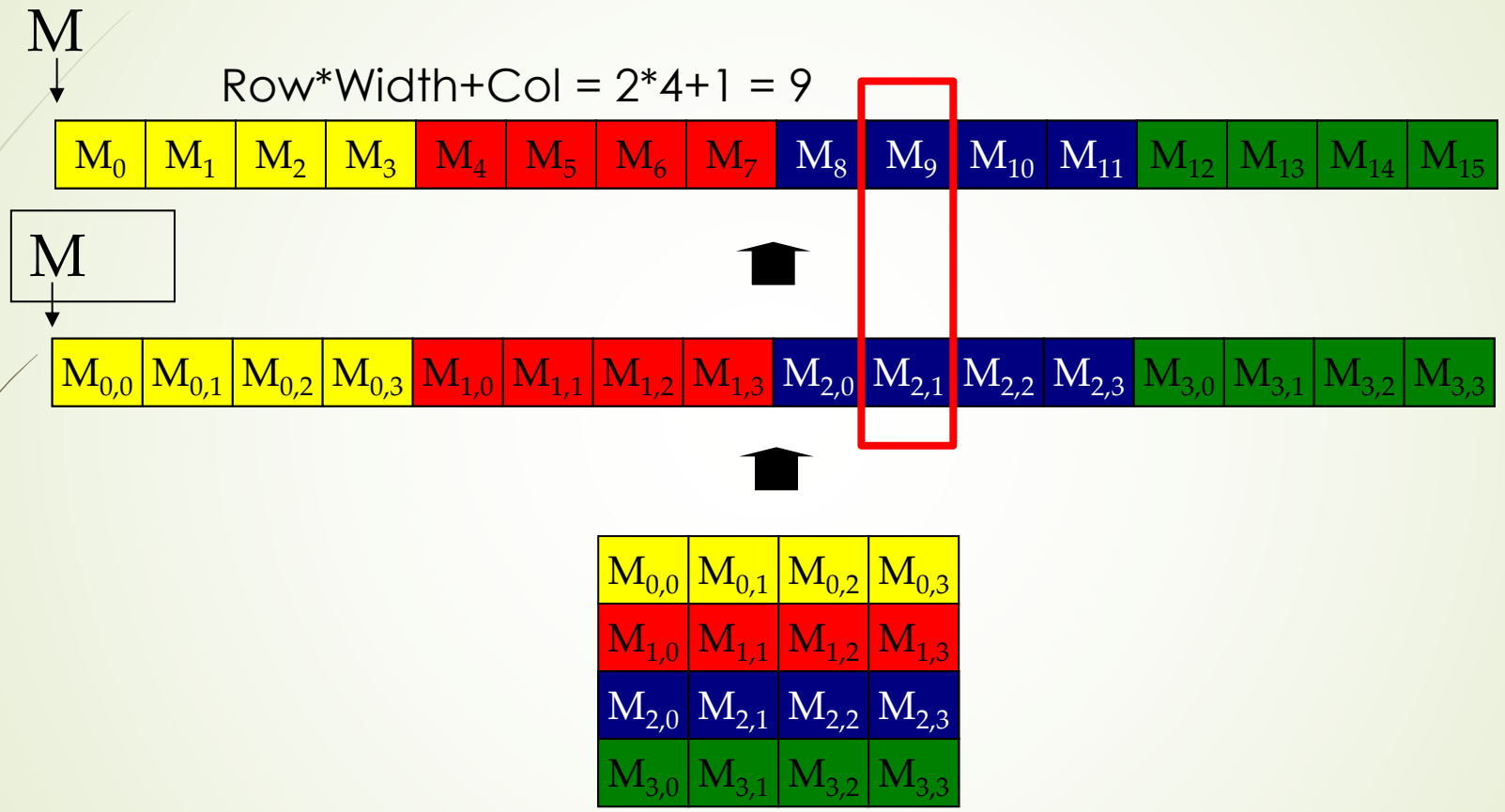


M = RxC matrix
M(row,col) = row*C + col ***row-major***
M(row,col) = col*R + row *col-major*

- How does this translate into 3D…?

  M(z,y,x) = M(slice,row,col)

Row-Major Layout in C/C++



Note that this is mathematically identical to forming the 1D globally unique thread ID using the formalism:

```
gidx = blockDim.x * blockIdx.x + threadIdx.x
```

CUDA linear memory data regions

CUDA provides a generic/arbitrary 1D memory array (contiguous bytes)

```
// allocate size bytes of linear (global, GMEM) memory on the device
__host__ cudaError_t cudaMalloc(void **devPtr, size_t size)
```

This is the primary way to exchange data between host and device GMEM

- *except for some specialized cases, which will discuss later: images, textures*
  - *These specialized types are called **CUDA Arrays***

- CUDA linear memory : arbitrary linear (1D) array of memory bytes

```
float* h_pData = (float*)malloc(sizeof(float)*N);    // alloc memory on host
cudaMalloc( &d_pData, sizeof(float)*N);              // alloc memory on device
cudaMemcpy( d_pData, h_pData, sizeof(float)*N, cudaMemcpyHostToDevice): // copy HtoD
```

Compute index space dimensionality

- There is a direct parallel to the application data – memory space relation
- The computational index space is also flattened/linearized when we calculate a gpu kernel (thread) global (linear) index
  - this is mathematically identical to finding the offset into a flattened/linearized multi-dimensional array (matrix, tensor, …)
- CONCEPTs
  - the total index space is the grid size : can be 1D, 2D, or 3D
    - 2D/3D host API & device (cuda) built-in variables
      - dim3 numblocksInGrid(Nx, Ny, 1);
      - dim3 numthreadsInBlock(Tx,Ty, Tz);
      - threadIdx.x, threadIdx.y, threadIdx.z
      - blockDim.x, blockDim.y, .z
      - blockIdx.x, blockIdx.y , .z
      - gridDim.x, .y, .z
      - warpSize
  - Int GlobalThreadIdxFrom1D = blockDim.x * blockIdx.x + threadIdx.x
  - gridSize.*i* = numblocksInGrid.*i* * numthreadsInBlock.*i*

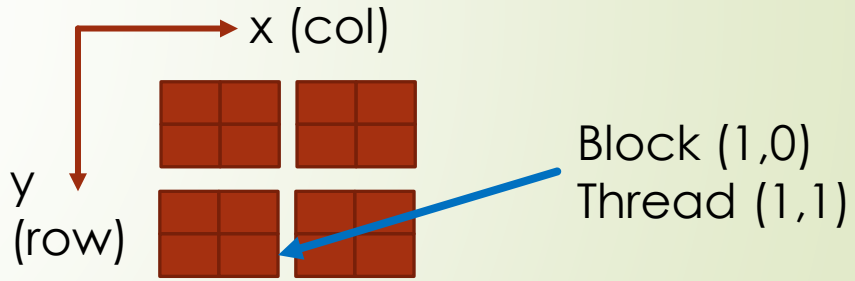
EXAMPLES

- 1D:
  - Num Blocks = 4, Block size = 4 (number of threads)



1DglobalLinearID = blockDim.x * blockIdx.x + threadIdx.x

- 2D:
  - Num Blocks = (2,2)
    - = gridDim.x,y
  - Threads per block= (2,2)
    - = blockDim.x,y



2DglobalLinearID = gridDim.x*blockDim.x*(blockDim.y*blockIdx.y + threadIdx.y)
+ blockDim.x*blockIdx.x + threadIdx.x

1D/2D/3D Grid Indexing

Assuming regular, uniform 1D/2D/3D grid

WIDTH = 2D slice width

HEIGHT = 2D slice height

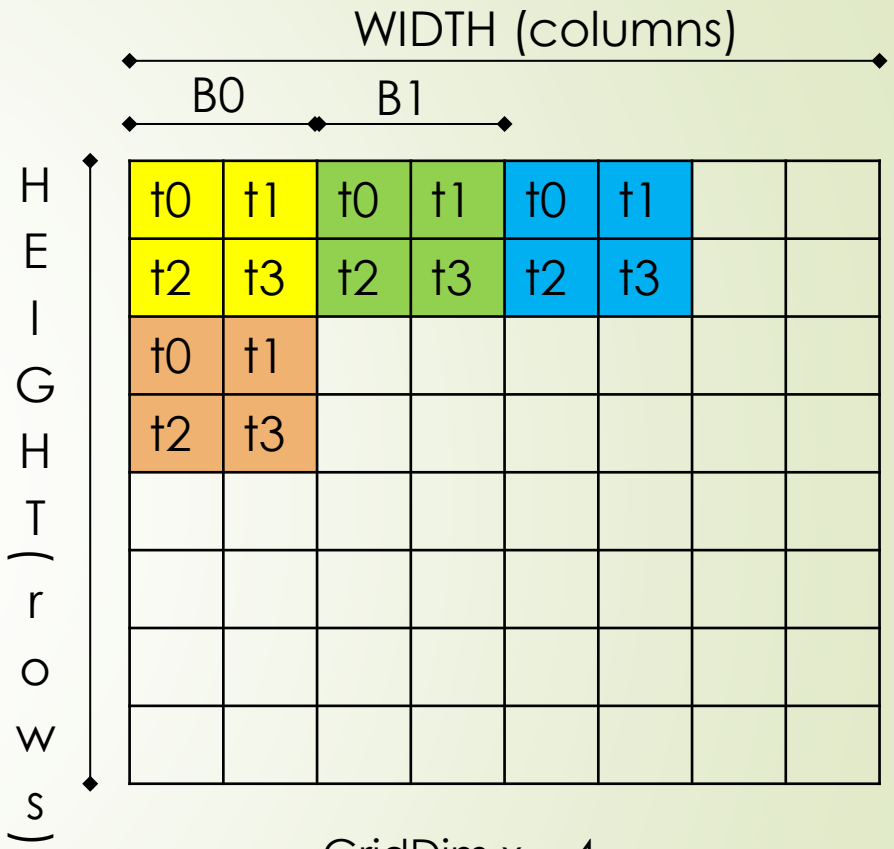
- 1D: x
- 2D: WIDTH*y + x
- 3D: WIDTH*HEIGHT*z + WIDTH*y + x

In GPU Grids (OpenCL/CUDA) we add concepts of

- Workgroups/Blocks
- Workitems/Threads (Lanes)
- let
  - Wx = GridDim.x * BlockDim.x
  - Wy = GridDim.y * BlockDim.y
  - Dx/y = BlockDim.x/y * BlockIdx.x/y

Then

- 1Dgtid = Dx + tid.x
- 2Dgtid = Wx*(Dy +tid.y) + (Dx + tid.x)
- 3Dgtid = Wx*Wy*(Dz + tid.z) + Wx*(Dy +tid.y) + (Dx + tid.x)

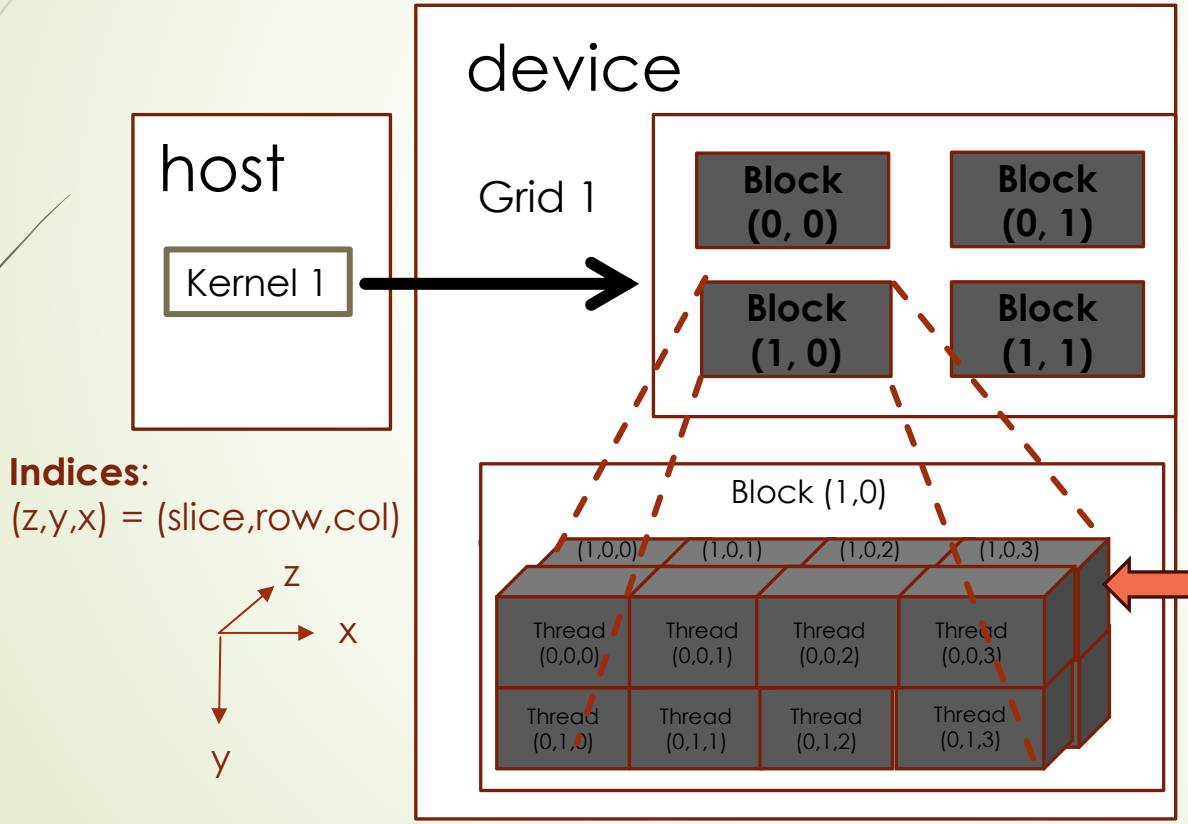


GridDim.x = 4
GridDim.y = 4
BlockDim.x = 2
BlockDim.y = 2

A Multi-Dimensional Grid Example

CUDA Grid/Block/Thread Hierarchy

– establishes a kernel thread's position in "index space" at runtime




```
// host code to create this grid
dim3 dimGrid(2,2,1);
dim3 dimBlock(4,2,2);
kernelfunction<<<dimGrid, dimBlock>>>(…);
```

```
// in cuda kernel function (device-side)
// obtain instance indices

// built-in block indices
int y_bidx = blockIdx.y  // 1
int x_bidx = blockIdx.x  // 0

// built-in thread indices
int z_tidx = threadIdx.z  // 1
int y_tidx = threadIdx.y  // 0
int x_tidx = threadIdx.x  // 3
```

– NOTE: as shown above, it is possible to have different dimensions for blocks/grid and threads/block, e.g. 2D grid with 3D threads.

Different (but related) spaces

Conceptually, we can think of three distinct but related spaces.

They can each have their own dimensionality and/or units.

In practice, they are all related.

1. Application-space problem size
2. Computational index space
   - related to how we decompose the problem for parallelization
   - and also to HW resources…
3. Hardware execution capabilities/resources
   - We'll get to these soon!

CUDA Grid: 2D Example

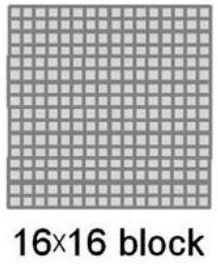
- With Boundary Handling
  - In general the size of the compute domain may not be evenly divisible by the block (and warp) dimensions.

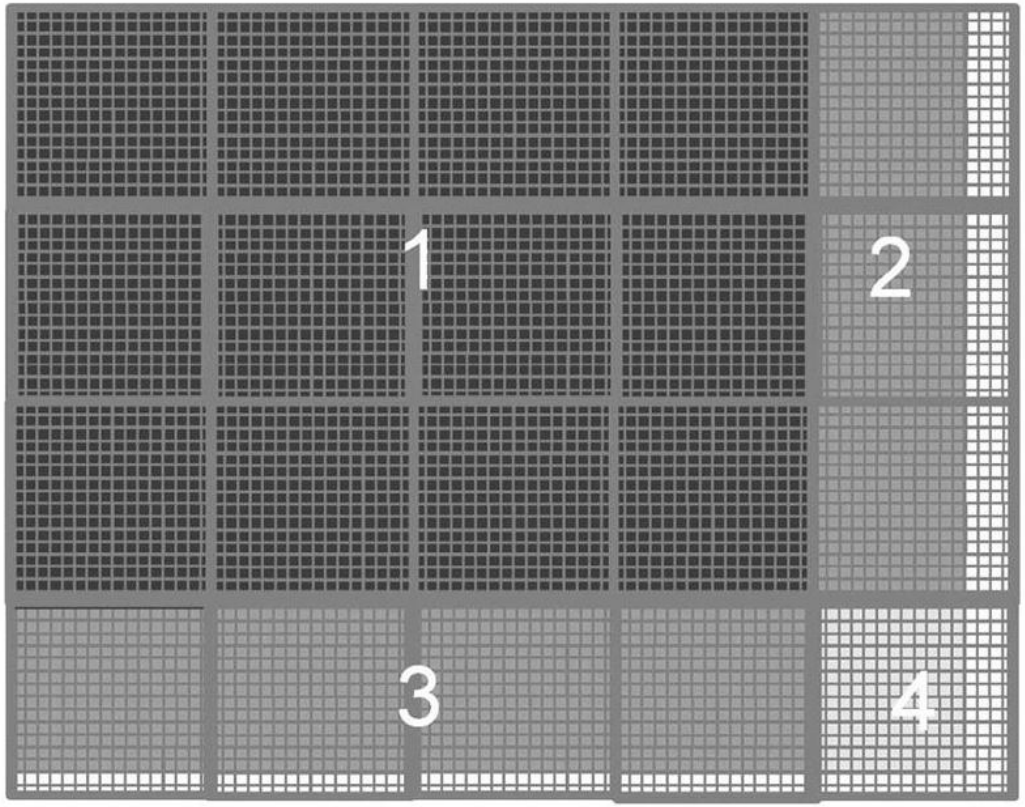
Example

- Suppose we want to process an image of size (MxN) = 62 x 76
- If we use following

```
dim3 dimGrid(ceil(m/16.0), ceil(n/16.0), 1);
dim3 dimBlock(16, 16, 1);  // threads per block
```

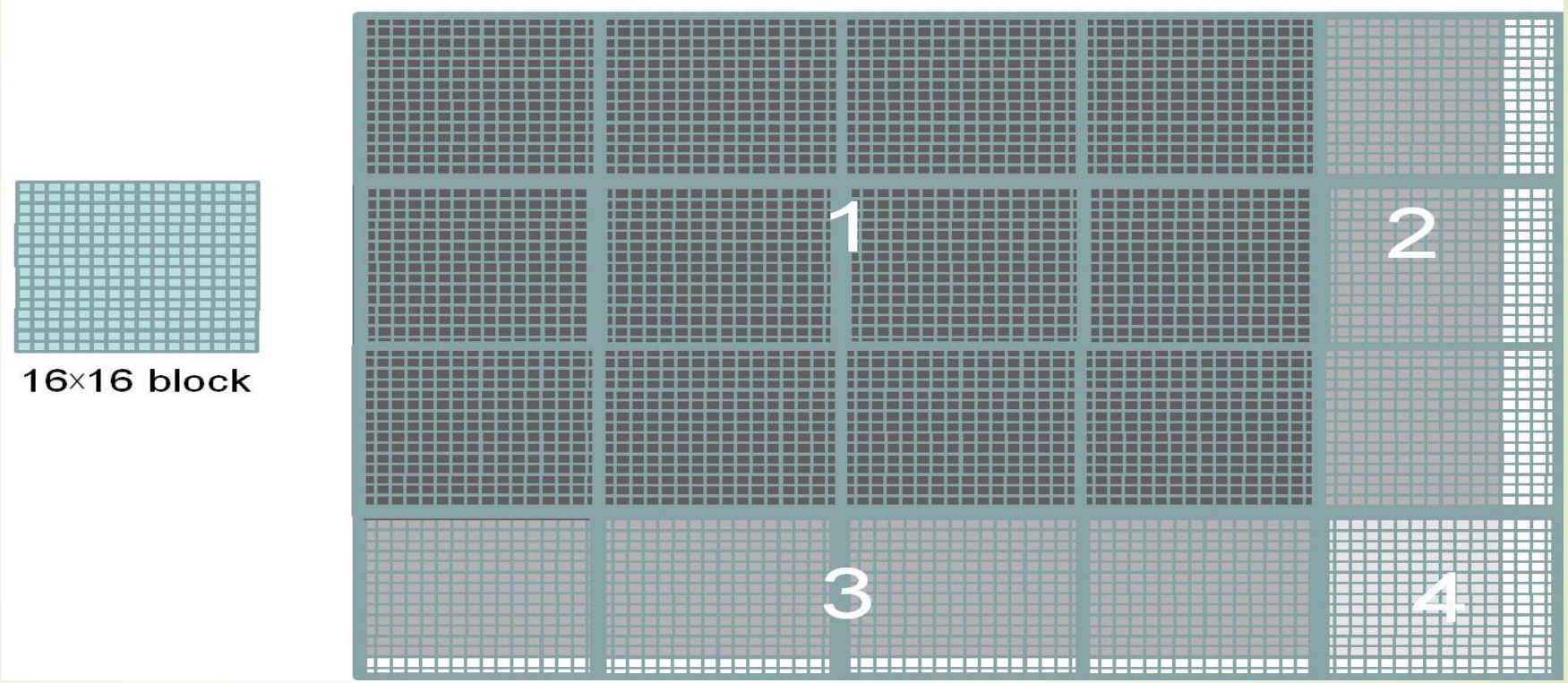
- We'll have a grid size of
  - 64x80 threads
  - 4x5 blocks





So there will be regions where the kernel will need to explicitly guard against out-of-bound accesses by some threads.

Covering a 62×76 Picture with 16×16 Blocks



Not all threads in a Block will follow the same **control flow** path.

We will soon see that the threads in group 2,3,4 will have significantly different performance behavior during execution, due to warp control flow divergence.

Color To Grayscale Conversion

- simplest case of mapping 2D threads to data (pixels)
- each RGB pixel input => output Grayscale pixel
  - is completely independent
  - pixel 'owned' by corresponding 2D thread
  - no overlaps or potential for data or memory side effects or interference

  - $Grey = 0.21 * R + 0.71 * G + 0.08 * B$

```
01  // The input image is encoded as unsigned chars [0, 255]
02  // Each pixel is 3 consecutive chars for the 3 channels (RGB)
03  __global__
04  void colortoGrayscaleConvertion(unsigned char * Pout,
05                    unsigned char * Pin, int width, int height) {
06      int col = blockIdx.x*blockDim.x + threadIdx.x;
07      int row = blockIdx.y*blockDim.y + threadIdx.y;
08      if (col < width && row < height) {
09          // Get 1D offset for the grayscale image
10          int grayOffset = row*width + col;
11          // One can think of the RGB image having CHANNEL
12          // times more columns than the gray scale image
13          int rgbOffset = grayOffset*CHANNELS;
14          unsigned char r = Pin[rgbOffset    ]; // Red value
15          unsigned char g = Pin[rgbOffset + 1]; // Green value
16          unsigned char b = Pin[rgbOffset + 2]; // Blue value
17          // Perform the rescaling and store it
18          // We multiply by floating point constants
19          Pout[grayOffset] = (unsigned char) 0.21f*r+0.71f*g+0.07f*b;
20      }
21  }
```

Parallel CUDA Image Blurring

INPUT



RESULT

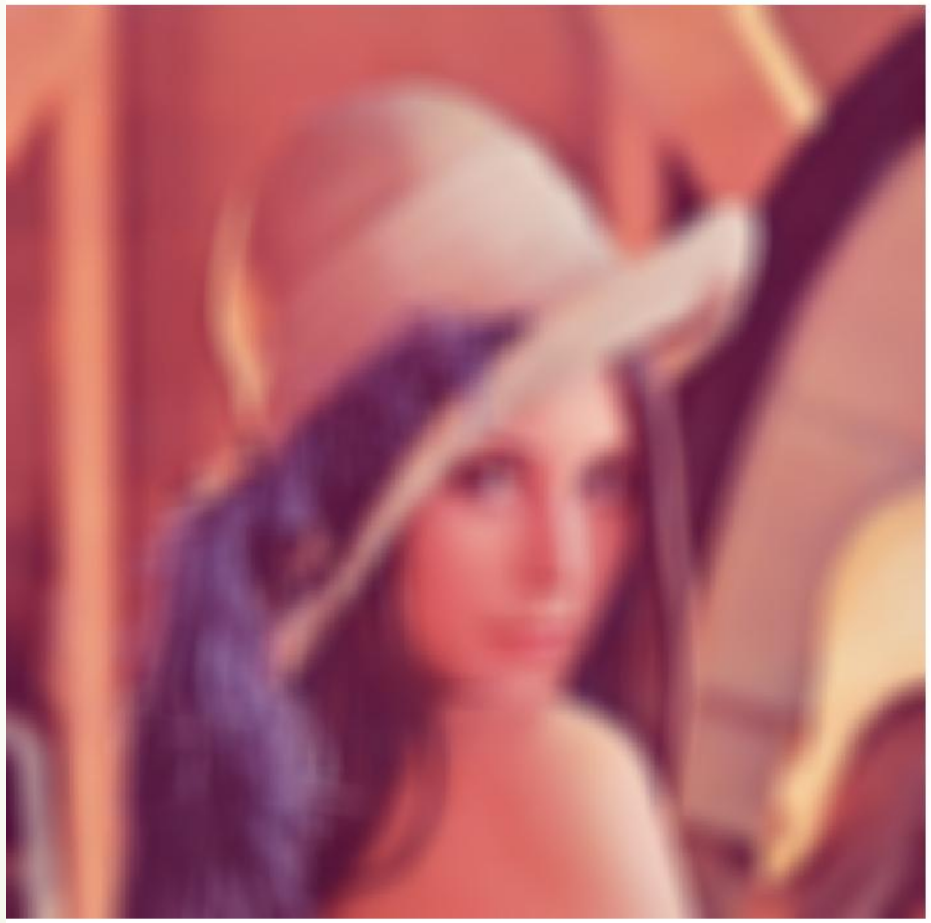
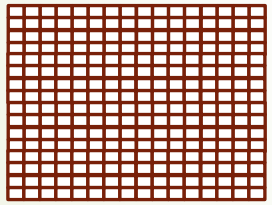


Photo credit: Dwight Hooker, 1972, "Lena"

Blurring Box

Blurring is accomplished by sweeping a ‘blur kernel’ or ‘blur filter’ (a small matrix) across a larger image.
The blur filter matrix coefficients act as a low-pass filter for a local neighborhood of pixels.
This is fundamentally an image convolution operation.

Where are the opportunities for parallelism?



Output Pixels processed by a thread block

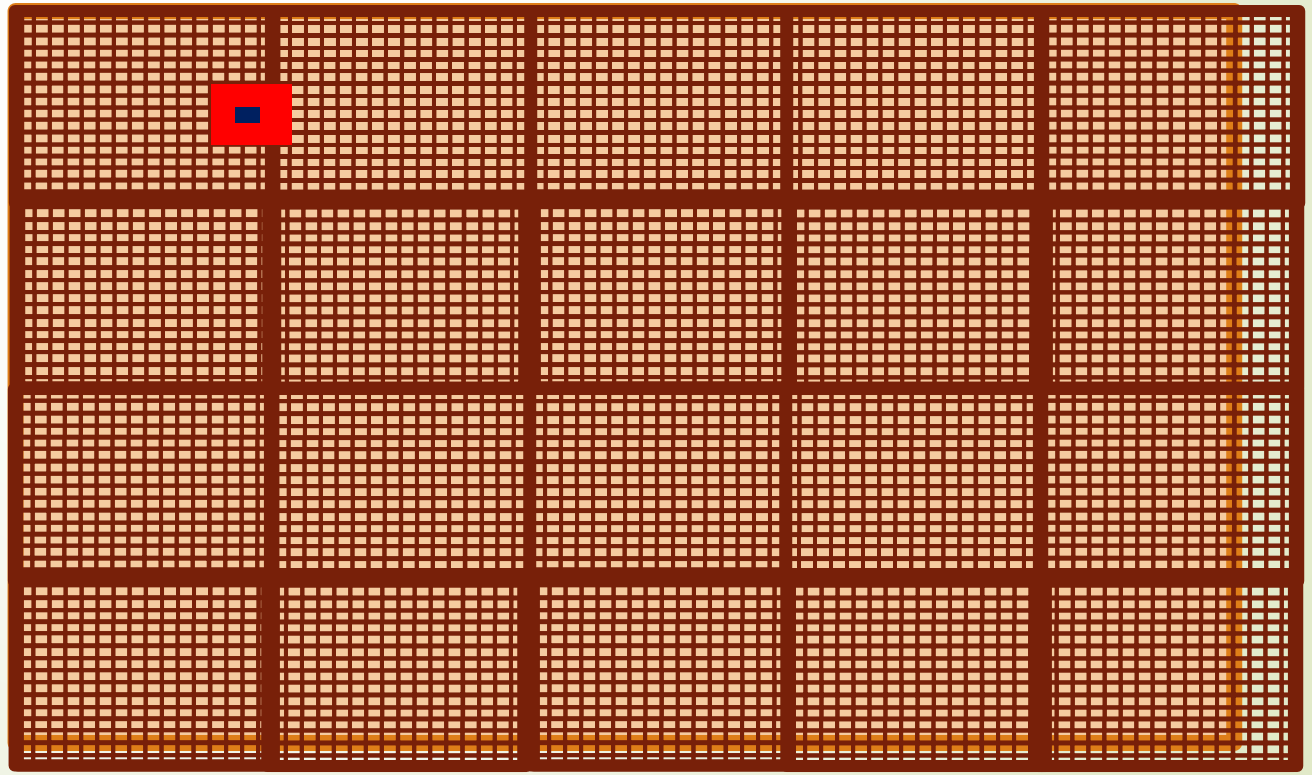


Image Blur as a 2D CUDA Kernel

```
__global__
void blurKernel(unsigned char * in, unsigned char * out, int w, int h)
{
    int Col  = blockIdx.x * blockDim.x + threadIdx.x;
    int Row  = blockIdx.y * blockDim.y + threadIdx.y;

    if (Col < w && Row < h) {
        ... // per-thread convolution operation goes here
    }
}
```

```
__global__
void blurKernel unsigned char * in, unsigned char * out, int w, int h, int BLUR_SIZE) {
   int Col  = blockIdx.x * blockDim.x + threadIdx.x;
   int Row  = blockIdx.y * blockDim.y + threadIdx.y;

   if (Col < w && Row < h) {
      int pixVal = 0;
      int pixels = 0;

      // Get the average of the surrounding 2xBLUR_SIZE+1 x 2xBLUR_SIZE+1 box
      for int blurRow = -BLUR_SIZE; blurRow < BLUR_SIZE+1; ++blurRow) {
         for int blurCol = -BLUR_SIZE; blurCol < BLUR_SIZE+1; ++blurCol) {

            int curRow = Row + blurRow;
            int curCol = Col + blurCol;
            // Verify we have a valid image pixel
            if (curRow > -1 && curRow < h && curCol > -1 && curCol < w) {
               pixVal += in[curRow * w + curCol];
               pixels++; // Keep track of number of pixels in the accumulated total
            }
         }
      }

      // Write new pixel value out
      out[Row * w + Col] = (unsigned char)(pixVal / pixels);
   }
}
```

HOST Application
CUDA Runtime API
Explicit memory mgmt

```
void explicitMemExample(int vectorLength)
{
    // Pointers for host memory
    float* A = nullptr; float* B = nullptr; float* C = nullptr;

    // Pointers for device memory
    float* devA = nullptr; float* devB = nullptr; float* devC = nullptr;

    //Allocate Host Memory using cudaMallocHost API. This is best practice
    // when buffers will be used for copies between CPU and GPU memory
    cudaMallocHost(&A, vectorLength*sizeof(float));
    cudaMallocHost(&B, vectorLength*sizeof(float));
    cudaMallocHost(&C, vectorLength*sizeof(float));

    // Initialize vectors on the host
    initArray(A, vectorLength);
    initArray(B, vectorLength);

    // Allocate memory on the GPU
    cudaMalloc(&devA, vectorLength*sizeof(float));
    cudaMalloc(&devB, vectorLength*sizeof(float));
    cudaMalloc(&devC, vectorLength*sizeof(float));

    // Copy data to the GPU
    cudaMemcpy(devA, A, vectorLength*sizeof(float), cudaMemcpyDefault);
    cudaMemcpy(devB, B, vectorLength*sizeof(float), cudaMemcpyDefault);
    cudaMemset(devC, 0, vectorLength*sizeof(float));
    // end-allocate-and-copy

    // Launch the kernel

    // Copy results back to host
    cudaMemcpy(C, devC, vectorLength*sizeof(float), cudaMemcpyDefault);

    // clean up
    cudaFree(devA); cudaFree(devB); cudaFree(devC);
    cudaFreeHost(A); cudaFreeHost(B); cudaFreeHost(C);
}
```

CUDA Host API - Kernel Launch

- Also called kernel *dispatch*
- uses 'triple chevron' <<<...>>> notation

```
// preceding d_A, d_B, d_C allocations and HtoD copies omitted
// launch grid of ceil(n/256.0) blocks of 256 threads each

// dim3 gridDim (.x, .y, .z)          - number of blocks in the grid
// dim3 blockDim (.x, .y, .z)         - block index within grid

dim3 DimGrid((n-1)/256 + 1, 1, 1);
dim3 DimBlock(256, 1, 1);

// example using dim3 (3D) launch configuration
//
vecAddKernel<<<DimGrid,DimBlock>>>(d_A, d_B, d_C, n);

// example using scalar 1D launch configuration
vecAddKernel<<<ceil(n/256.0),256>>>(d_A, d_B, d_C, n);
```

Examples: Host Code for Kernel Launches (1D, 2D)

```
// 1D vector addition : Launch the kernel
int threads = 256;
int blocks = cuda::ceil_div(vectorLength, threads);
vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
// wait for async kernel execution to complete
cudaDeviceSynchronize();


// 2D Blur Kernel Launch
// assume that the picture is m × n (rows x columns)
// m pixels in y dimension (height, rows)
// n pixels in x dimension (width, columns)

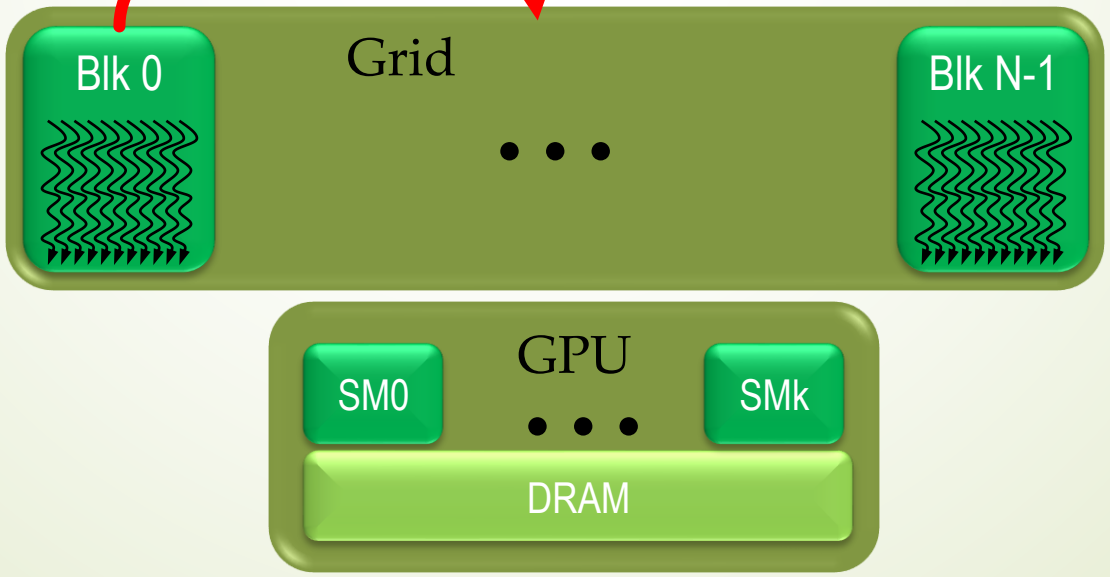
// input d_Pin has been allocated on host and copied to device (HtoD)
// output d_Pout has been allocated on device
…
dim3 DimGrid((n-1)/16 + 1, (m-1)/16+1, 1);
dim3 DimBlock(16, 16, 1);
blurKernel<<<DimGrid,DimBlock>>>(d_Pin, d_Pout, m, n);
…
```

Kernel execution (dispatch) "in a nutshell"

```
__host__
void vecAdd(…)
{
  dim3 DimGrid(ceil(n/256.0),1,1);
  dim3 DimBlock(256,1,1);
  vecAddKernel<<<DimGrid,DimBlock>>>(d_A,d_B,d_C,n);
}
```

```
__global__
void vecAddKernel(float *A,
     float *B, float *C, int n)
{
   int i = blockIdx.x * blockDim.x
       + threadIdx.x;

   if( i<n ) C[i] = A[i]+B[i];
}
```



3D EXAMPLE

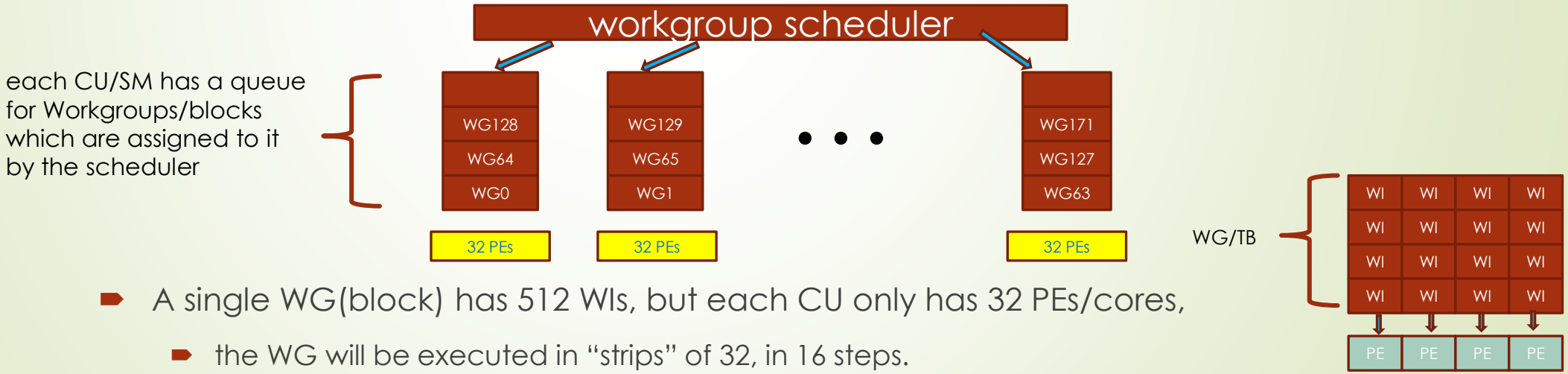
- 3D MRI volume medical imaging
  - Application-space: 3D physical coordinates (x,y,z)
    - device sampling resolution provides 1001x1001x511 volumetric dataset
  - Computational index space:
    - decompose/partition full 3D grid into work units
      - N threadblocks, M threads/block
    - these choices are guided by
      - the specific parallelization algorithm being implemented
      - HW resources : device SMs and cores, device memory (global, shared/local, private)
      - choices tend to use **powers-of-two**.
    - We'll pick closest power-of-two: 1024x1024x512 = 128*8 x 128*8 x 64*8
      - so NumThreadBlocks = 128 x 128 x 64 = 1,048,576 = 1024 Ki
      - and Threads = 8 x 8 x 8 = 512

CUDA

```
dim3 numBlocks{ 128, 128, 64 };
dim3 threadsPerBlock{ 8, 8, 8 };
myKnl<<<numBlocks,threadsPerBlock>>>(a,b,c)
```

3D EXAMPLE (cont…)

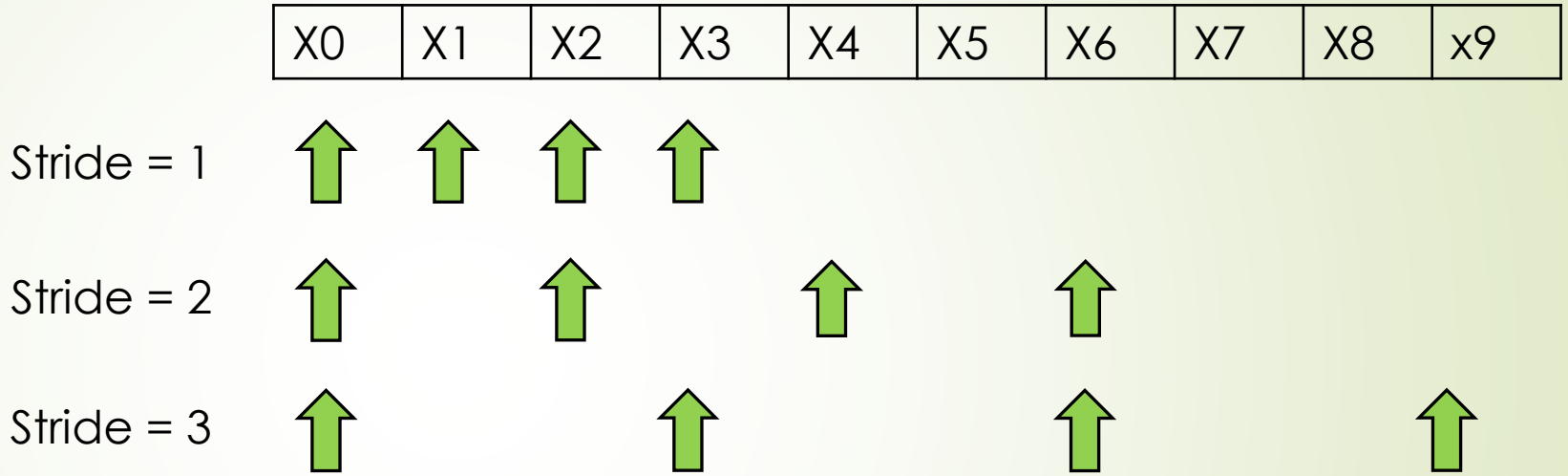
- We started with 1001x1001x511 volumetric MRI data
- With a compute index space of: 1024x1024x512 = 128*8 x 128*8 x 64*8
  - Number of WorkGroups (threadblocks) : 128 x 128 x 64 = 1,048,576 (= 1024 Ki)
  - Number of WorkItems / (threads) size: 8 x 8 x 8 = 512
- SUPPOSE we have a Hypothetical GPU
  - **64** CUs (SMs) with **32** PEs (Processing Elements = CUDA cores) each = 2048 cores total
    - We'll also assume a single partition/SM, while real NVIDIA devices currently have 4 sub-partitions/SM



- A single WG(block) has 512 WIs, but each CU only has 32 PEs/cores,
  - the WG will be executed in "strips" of 32, in 16 steps.
    - each strip of 32 threads is a ***warp*** (CUDA term)
    - the full set of threadblocks executing simultaneously across device is called a ***wave*** (NVIDIA CUDA term)

Grid/NDRange Additional Considerations

- Strides and striding : 1D data len N = 10. gridsize = 1 block, blocksize = 4 threads



- Suppose our total data size N is larger than the total grid size
  - We can have kernels loop where each thread strides by the grid size
    - "**Grid-stride loop**" formalism with data boundary handling:

```
__global__ void vecAdd_gridstride(float *result, float *a, float *b, int N) {
  int index = threadIdx.x + blockIdx.x * blockDim.x;
  int stride = blockDim.x * gridDim.x;

  for(int i = index; i < N; i += stride)
  {
     result[i] = a[i] + b[i];
  }
}
```

Determining Grid Sizing

- There are ways to automatically determine a reasonable (not always optimal) size for your execution configuration
  - blocks-per-grid
  - threads-per-block
- Rules-of-Thumb (*sometimes broken*)
  - Total threads: goal is 512+ threads per SM (aim for max occupancy supported)
  - threads-per-block = even multiple of:
    - warpsize (32)
    - total # SMs on device
    - Consider HW device resource limits
    - 128-256 threads/block is often good starting point
  - Total grid size =
    - Large enough to include full problem computational domain
    - Consider device resources and limitations, warpsize

```
int threadsPerBlock = 128;
int blocksPerGrid = (numElements + threadsPerBlock - 1) / threadsPerBlock;
// OR
int blocksPerGrid = ceil(numElements/ threadsPerBlock);
my1Dkernel<<<blocksPerGrid,threadsPerBlock>>>(foo,bar);
```

CUDA Programming: Host & Device

- **Host Code:** Application APIs
  - CUDA Runtime API
    - provides both an API and some language extensions to handle common tasks such as
      - memory allocation routines,
      - copying data between GPUs and other GPUs or CPUs,
      - simplified launching of kernels.
    - Implemented on top of lower-level CUDA Driver API
  - CUDA Driver API
    - fine-grained control over
      - explicit kernel and module loading
      - explicit context management (for larger SW package integration)

- **Device Code:** CUDA device C++ language extensions…

CUDA C++ Device Kernel Language - Data Types

Built-in vector data types

vector (*n* in {1,2,3,4})

- char*n*, uchar*n*
- short*n*, ushort*n*
- int*n*, uint*n*
- long*n*, ulong*n*
- float*n*, double*n*

struct dim3

- integer vector type
- used to specify dimensions
- any unspecified component is initialized to 1

physical sizes of common CUDA scalar data types

- char: 1 byte
- short: 2 bytes
- int: 4 bytes
- long: 8 bytes
- float: 4 bytes
- double: 8 bytes
- bool: 1 byte

For vector types, the sizes are as follows:

- float2: 8 bytes (2 floats)
- float3: 12 bytes (3 floats)
- float4: 16 bytes (4 floats)
- double2: 16 bytes (2 doubles)
- double3: 24 bytes (3 doubles)
- double4: 32 bytes (4 doubles)

Sizes are consistent with standard C++ data types
NOTE: there are alignment requirements

SOME EXAMPLES:

```
// auto-determine number of thread blocks needed based on specified blocksize and data size (Nx, Ny, Nz)
// standard formalism
dim3 threads_per_block(8,8,1);
dim3 number_of_blocks( (Nx+threads_per_block.x -1)/threads_per_block.x, (Ny+threads_per_block.y -1)/threads_per_block.y, 1);
```

```
float4 f4 = make_float4(1,2,3,4);  // they all come with constructor function of form make_<type name>
float f0 = f4.x;  // access (x,y,z,w) field components
float f3 = f4.w;
```

- derived from the basic integer and floating-point types.
- They are structures and the 1st, 2nd, 3rd, and 4th components are accessible through the fields x, y, z, and w

CUDA C++ Kernel Language - Function and Memory

function execution space specifiers

- where function executes, who it's callable by

	Executed on the:	Only callable from the:
`__device__ float DeviceFunc()`	device	device
`__global__ void KernelFunc()`	device	host
`__host__ float HostFunc()`	host	host

`__global__` defines a kernel function

- Each "__" consists of two underscore characters
- A kernel function must return `void`

`__device__` and `__host__` can be used together (code will compile for each context)

`__host__` is optional if used alone

- this is also the default if no declaration is provided

variable memory space qualifiers

- denote memory location on device of a variable

```
__device__
__shared__
__constant__
```

CUDA C++ Device / Kernel Language –
Built-in variables and functions

Built-in variables
these can be access from any kernel code running on the device

- dim3 blockDim (.x, .y, .z) - number of threads in the block
- uint3 threadIdx (.x, .y, .z) - local thread index within the block
- int warpSize - warp size in threads (should be 32)

Built-in function extensions
There are many…
We will learn to use many of these as we proceed through the course

- Mathematical
- Memory Fence
- Atomic operations
- Synchronization
- Asynchronous Barrier, Data Copy

CUDA C++ Kernel Language - NVIDIA Documentation

For the definitive current documentation look here:

C/C++ Language Extensions

- https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#

C++ Language Support

- https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-support.html
- describes CUDA support for C++ 11/14/17/20/23 Standard features

Result verification: CPU v GPU

- Important part of analyzing your parallel computation implementation is checking you get the right answer, also called **verification**.
- When the same calculation is done on both CPU and GPU, the results may not be identical. Why?
  - different ordering of operations can affect results due to rounding, truncation/overflow
    - floating-point is not associative! (A+B)+C is not necessarily equal to A+(B+C)
  - numerical accuracy and precision of different hardware
  - particularly evident for floating-point calculations
    - IEEE 754 compliance and specific options (rounding modes, etc)

- THUS
  - common to use small error **tolerance** values for acceptable agreement threshold

```
float rTol = 1e-5;  // relative tolerance
float aTol = 1e-6;  // absolute tolerance
if( abs( cpuResult - gpuResult) <= max(aTol, rTol*max(abs(cpuResult),abs(gpuResult))) )
     correct = TRUE;
```

  - Recommendations
    - **FP32** rtol = 1e-5, atol = 1e-6
    - **FP64** rtol = 1e-12, atol = 1e-15

NVIDIA GPU generations microarchitecture families

- Two different product families:
  - Professional
    - ECC memory
  - Consumer (gaming)
    - No-ECC
    - Often include display interface HW

Microarchitecture	Compute Capability (CC)
Kepler	3.x
Maxwell	5.x
Pascal	6.x
Volta	7.x
Turing	7.5
Ampere	8.x
Ada Lovelace	8.9
Hopper	9.x
Blackwell	10.x

Professional Product	Graphics Cards
Blackwell (2024)	B100 B200
Hopper (2022)	H100 H200
Ampere (2020)	A100
Volta (2017) (Pred. - Pascal)	Tesla V Titan V Quadro GV100

Consumer Product	Graphics Cards
Ada Lovelace (2022)	GeForce 40 series
Ampere (2020)	GeForce 30 series
Turing (2018)	GeForce 16 series GeForce 20 series
Pascal (2016)	GeForce 10 series Quadro P Tesla P4
Maxwell (2014)	GeForce 700 series GeForce 800M series GeForce 900 series
Kepler (2012)	GeForce 600 series GeForce 700 series GeForce 800M series
Fermi (2010)	GeForce 400 series GeForce 500 series
Tesla (2006)	GeForce 8 series GeForce 9 series GeForce 100 series GeForce 200 series GeForce 300 series

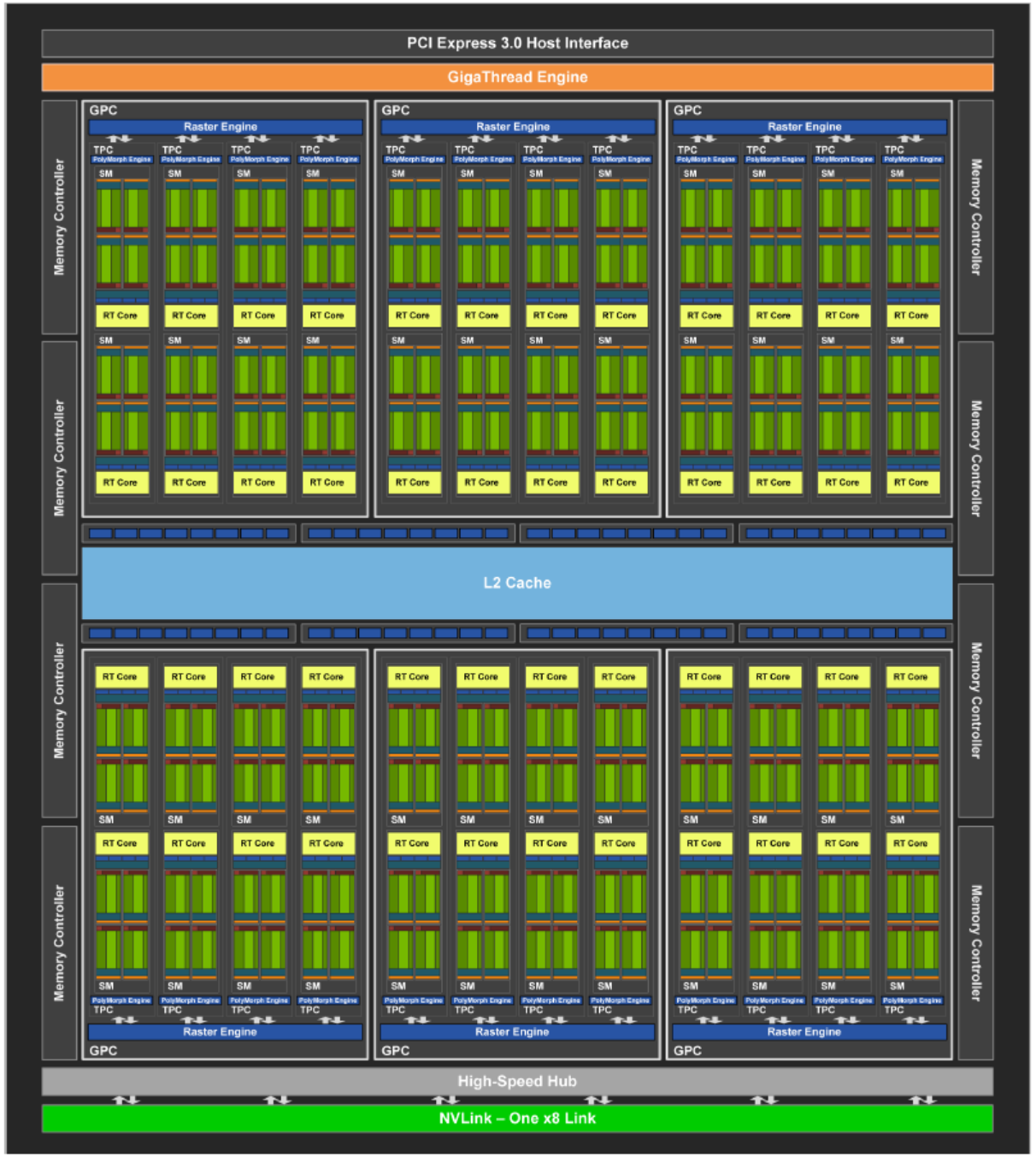
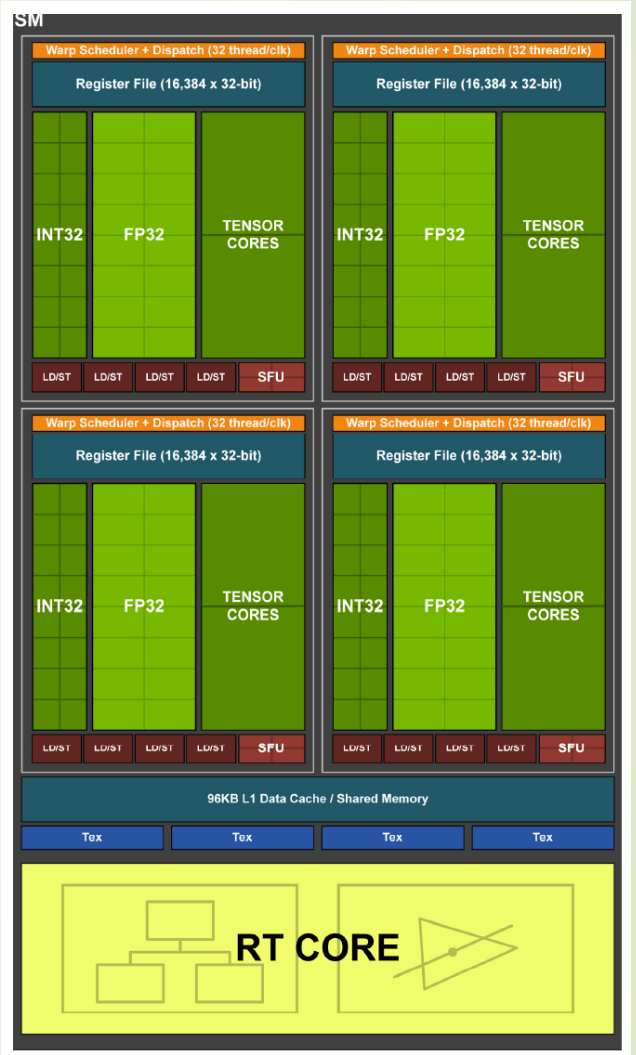


Figure 41. Turing TU104 Full Chip Diagram

Turing T4 (CC 7.5)

- Unified shared memory/L1cache per SM = 96KB
- 64KB/32KB or 32KB/64KB
- L2 = 6MB
- Each SM
  - 64 FP32 + 64 INT32 cores
    - parallel execution pipes
  - 4 process blks (partitions)
    - 16 FP32, 16 INT32 cores
    - 2 Tensor cores
    - 1 warp scheduler
    - 1 dispatch unit
    - L0 instruction cache
    - 64KB register file
- 1st GEN Ray Trace Cores
- 2nd Gen Tensor Cores
- Concurrent FP32 + INT32



Turing Tesla T4

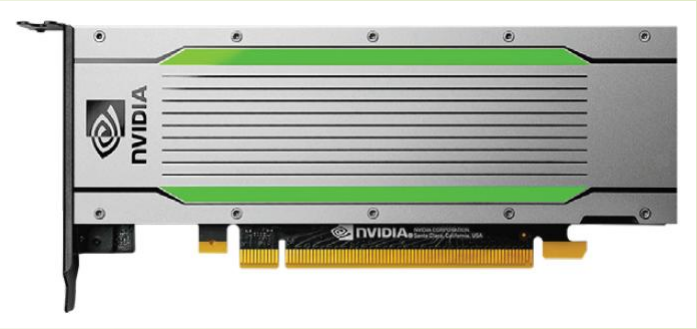


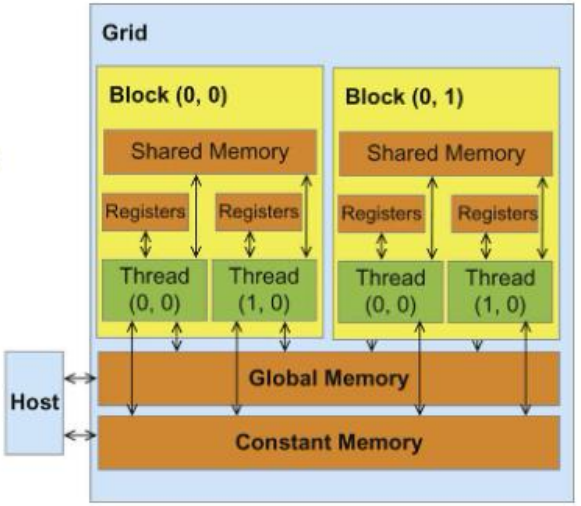
Table 5. Comparison of the Pascal Tesla P4 and the Turing Tesla T4

GPU	Tesla P4 (Pascal)	Tesla T4 (Turing)
GPCs	4	5
TPCs	20	20
SMs	20	40
CUDA Cores/SM	128	64
CUDA Cores/GPU	2,560	2,560
Tensor Cores/SM	NA	8
Tensor Cores/GPU	NA	320
RT Cores	NA	40
GPU Base Clock MHz	810	585*
GPU Boost Clock MHz	1,063	1,590
Peak FP32 TFLOPS	5.5	8.1
Peak INT32 TIPS	NA	8.1
Peak FP16 TFLOPS	NA	16.2
Peak FP16 Tensor TFLOPS with FP16 Accumulate*	NA	65
Peak FP16 Tensor TFLOPS with FP32 Accumulate*	NA	65
Peak INT8 Tensor TOPS*	22	130
Peak INT4 Tensor TOPS*	NA	260
Frame Buffer Memory Size and Type	8192 MB GDDR5X	16384 MB GDDR6
Memory Interface	256-bit	256-bit
Memory Clock (Data Rate)	6 Gbps	10 Gbps
Memory Bandwidth (GB/sec)	192	320
ROPs	64	64
TDP	75 Watts	70 Watts

- global mem: large & slow
- shared mem: small & fast

Device code can:
- R/W per-thread registers
- R/W per-thread local memory
- R/W per-block shared memory
- R/W per-grid global memory
- Read only per-grid constant memory

Host code can
- Transfer data to/from per grid global and constant memories



- use shared mem (SMEM) when the overhead of moving data from global can be offset by repeated use of the data within the block

Kernel hardware execution resources

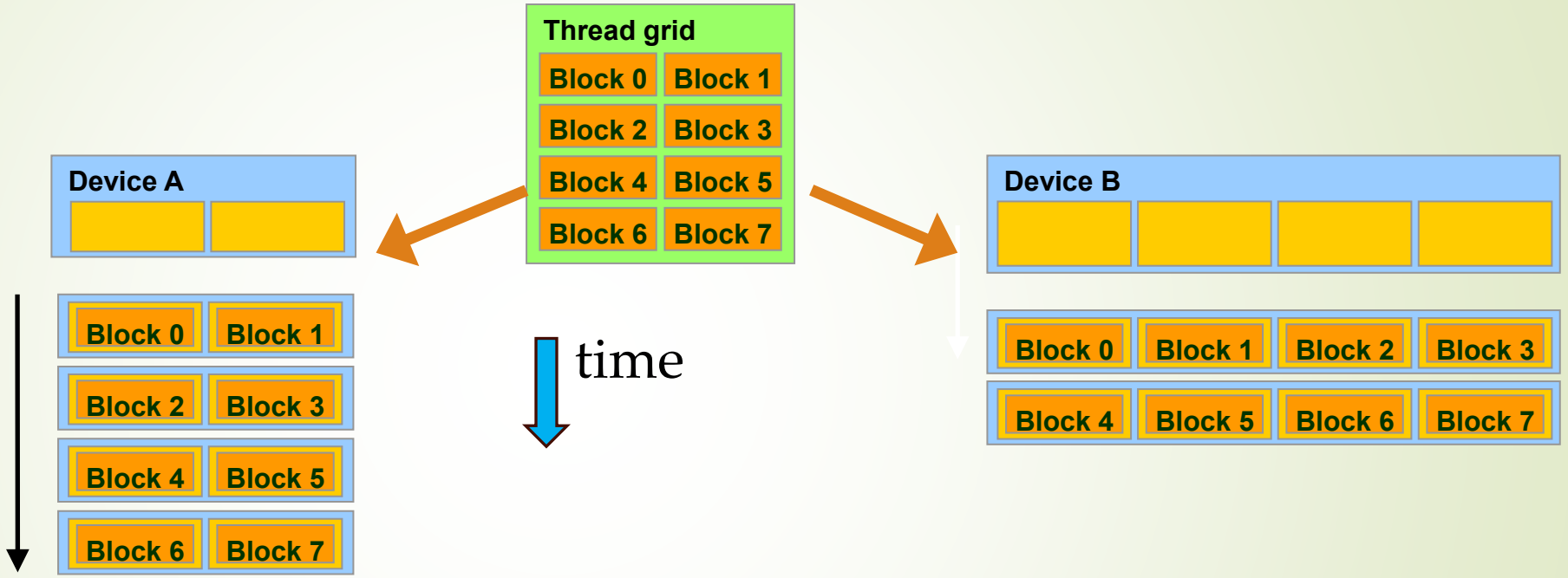
- Consider how your CUDA kernel utilizes hardware execution resources
  - Assigning thread blocks, threads to HW execution resources
  - Capacity constrains of HW execution resources

Turing Tesla T4 –(CC 7.5)

Hardware Execution Resources Limits

Max Block Dim (X/Y/Z) = #threads/block/dim	1024/1024/64
Max Blocks Per Multiprocessor (SM)	16
Max Grid Dim (X/Y/Z) = #blocks/grid/dim	2147483647/65535/65535
Max registers per block	65536
Max registers per SM	65536
Max Shared Memory per block (default)	49152
Max Shared Memory per SM	65536
Max threads per block	1024
Max threads per SM	1024
Multiprocessor (SM) Count	40
Warp size	32
Total constant memory	65536
S to D precision perf ration	32
(main) clock rate	1590000
Memory clock rate	5001000
global memory bus width	256
L2 cache size	4194304
Max persisting L2 cache size	0
ECC enabled	true

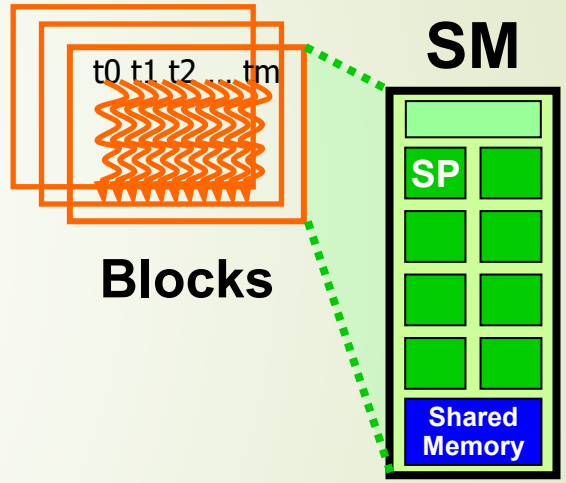
Transparent Scalability



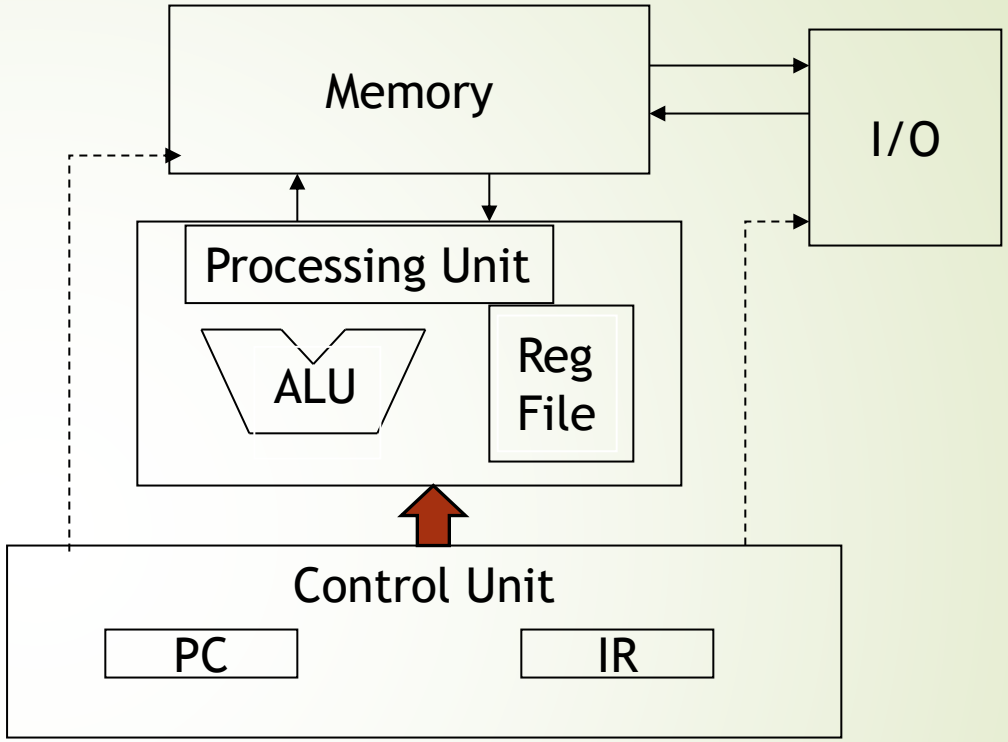
- Each block must be able to execute in any order relative to others.
- Thus hardware is free to assign blocks to any processor at any time
  - A cuda kernel scales ‘transparently’ to any number of parallel processors
  - Some automatic performance improvement with new generations…

Example: Allocating & Executing Thread Blocks

- Threads are assigned to Streaming Multiprocessors (SM)
- in block granularity
  - All warps in block will go to same SM
    - Up to 32 blocks to each SM (as resources allow)
    - Turing SM can take up to **16** blocks & **1024** threads
      - Could be 256 (threads/block) * 4 blocks
      - Or 512 (threads/block) * 2 blocks
      - etc.
- SM maintains thread/block idx #s
- SM manages/schedules warp (thread lanes) execution



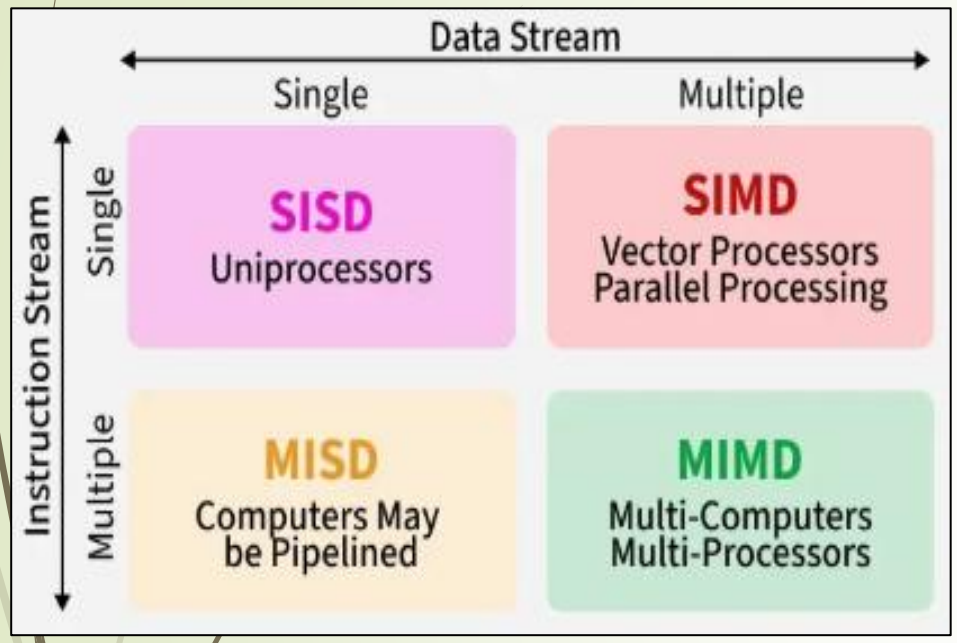
The Von-Neumann Model
computer architecture



Key Components

1. A processing unit with both an arithmetic logic unit (ALU) and processor registers
2. A control unit that includes an instruction register (IR) and a program counter (PC)
3. Memory that stores both data and instructions in same address space
4. External mass storage
5. Input and output mechanism

Flynn's Taxonomy

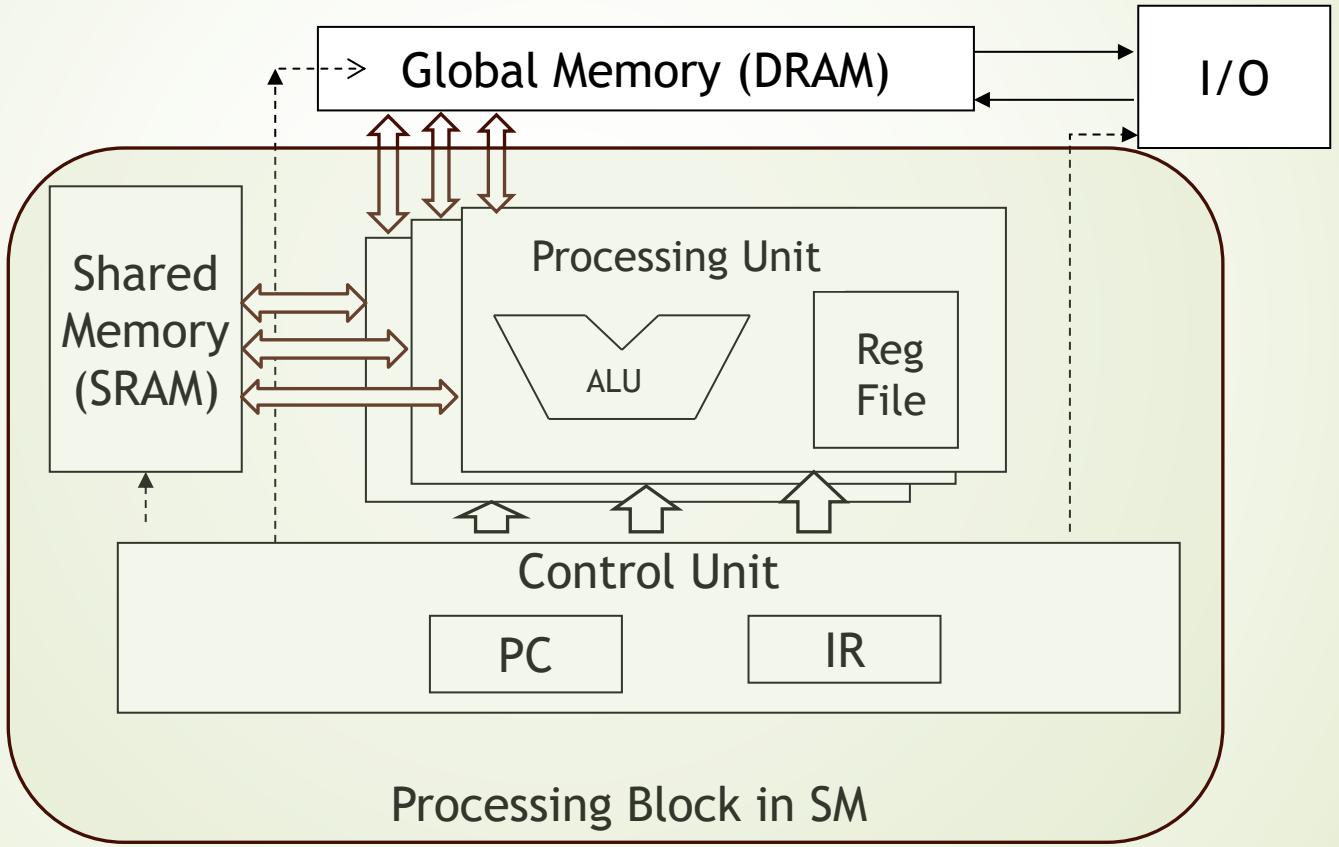


Classifications of Computer Architecture
Dr/Professor Michael J. Flynn, Stanford, 1966

- SISD (Single Instruction, Single Data)
- SIMD (Single Instruction, Multiple Data)
- MISD (Multiple Instruction, Single Data)
- MIMD (Single Instruction, Multiple Data)

The Von-Neumann Model with **SIMD** units

Single Instruction Multiple Data: (SIMD)



SIMD versus SIMT

SIMT = Single Instruction Multiple ***Threads*** (NVIDIA term)

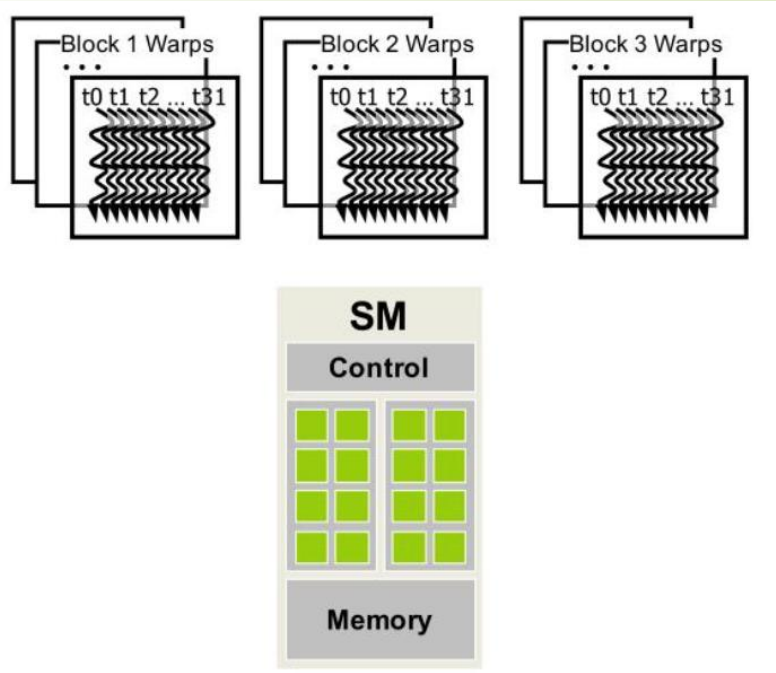
Both models run one instruction over many ALU lanes at once.
They differ in **what the programmer writes** and **what the hardware automatically**

CPU SIMD
the *vector* is the unit of programming:
one thread issues instructions on wide registers,
programmer (or compiler) explicitly manage width, masking, and data layout.

NVIDIA's SIMT
the *scalar thread* is the unit of programming:
programmer declares independence of cuda kernel threads,
the hardware groups 32 threads into a **warp** that runs on SIMD lanes. The hardware implicitly handles masking, divergence, per-lane addressing, and latency hiding at run time.

Warps as Scheduling Units

- Each Block is executed as **32-thread Warps**
  - Once a block is assigned to an SM, it is divided into 32-thread warps.
  - *An implementation decision, not part of the CUDA programming model*



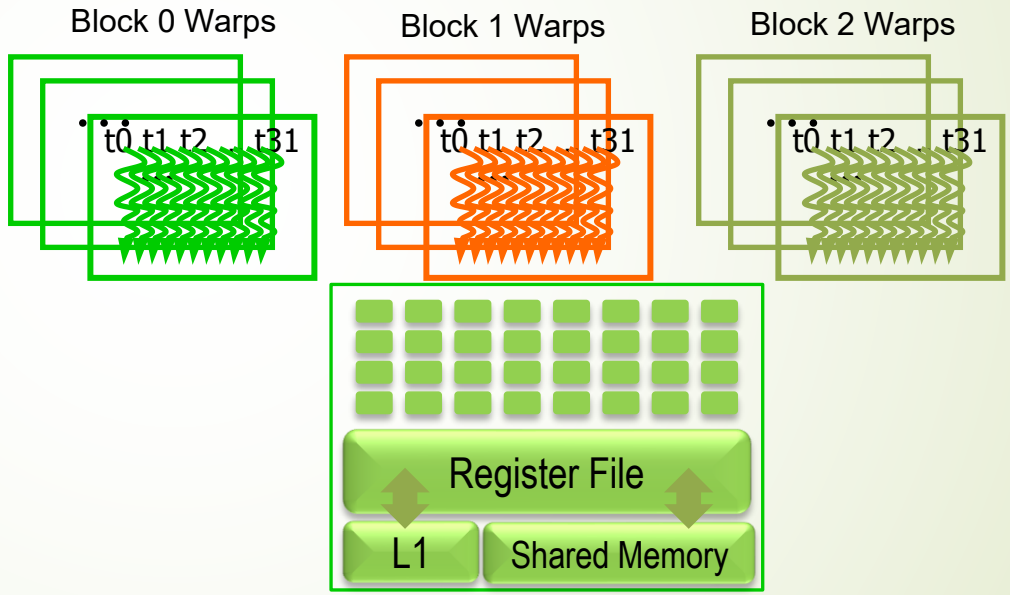
  - Warps are scheduling units in SM
  - Blocks are partitioned into warps based on consecutive increasing thread indices
    - For 1D grid
      - warp0 = threadIdx 0 – 31
      - warp1 = threadIdx 32 – 63
    - For 2D/3D grid
      - dims projected row-major order
  - Threads in a warp execute in SIMD “lanes”
  - Partial warps padded with inactive threads
  - Future GPUs may have different number of threads in each warp

Warp Examples

- If 3 blocks are assigned to an SM and each block has 256 threads, how many Warps are there in an SM?
  - Each Block is divided into 256/32 = 8 Warps
  - There are 8 * 3 = 24 Warps

blockIdx: 0…2

threadIdx: 0…31



- What will the threadIdx-to-warp mapping be for a 3D ZxYxX = 2x8x4 threadblock? Using T(z,y,x)
  - warp0 : thread indices T(0,0,0) … T(0,7,3)
  - warp1 : thread indices T(1,0,0) … T(1,7,3)

Example: Warp/Thread Scheduling (Cont.)

- SM implements **zero-overhead warp scheduling**
  - Warps whose next instruction have operands ready for consumption are eligible for execution
  - Eligible Warps are selected for execution based on a prioritized scheduling policy
  - All threads in a warp execute the same instruction when selected
    - SIMT model (allows writing scalar kernel code which uses SIMD HW)

- Warp-level primitives
  - collection of API functions which provide efficient data exchange and synchronization across threads in a warp
    - `Warp Vote / Match / Reduce / Shuffle / _ _sync`
  - Treat warps as an extra level in software hierarchy (between blocks and threads)
  - Can result in significant performance benefits

ThreadBlock Granularity Considerations

- For kernels using multiple blocks, should each block have 4X4, 8X8 or 30X30 threads for Volta?
  - NOTE: Each Volta SM can take up to 2048 threads, 32 blocks

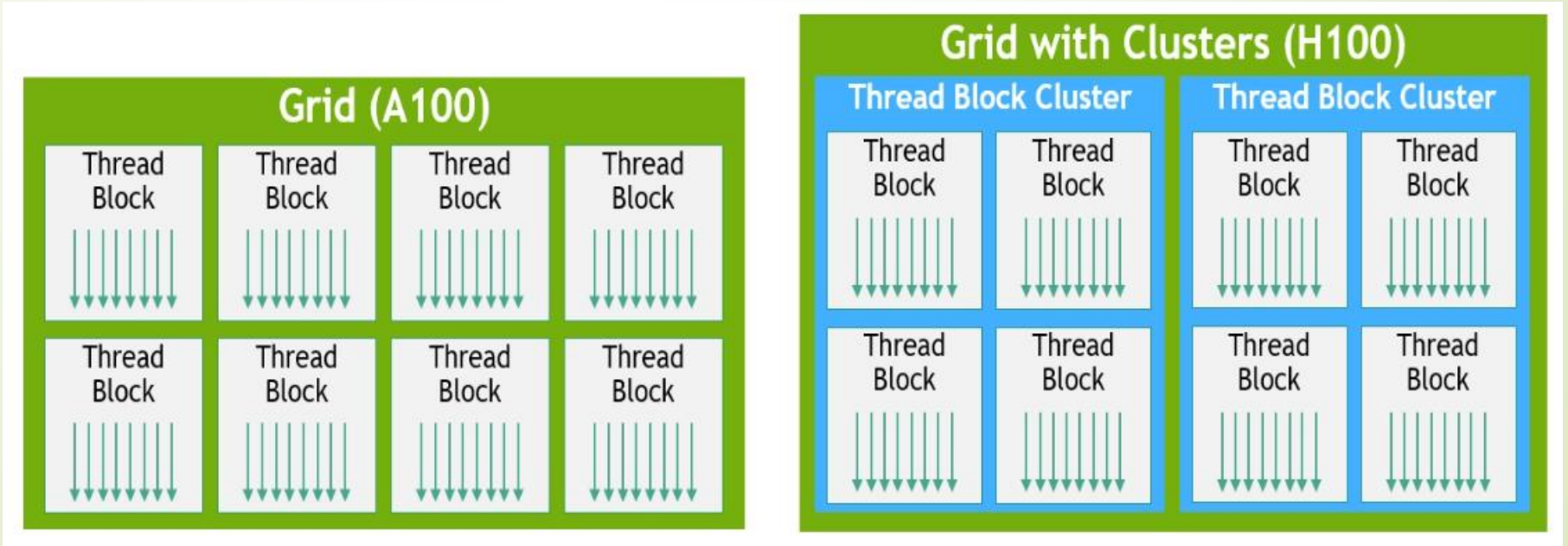
- For 4X4, we have 16 threads per Block.
  - Each SM can take up to 2048 threads: 2048/16 =128 Blocks.
  - But each SM can only take up to 32 Blocks,
  - so only 16*32 = 512 threads will go into each SM. 512/2048 = 25% utilization

- For 8X8, we have 64 threads per Block.
  - Since each SM can take up to 2048 threads, it can take up to 32 Blocks
  - Thus 32*64 = 2048
  - **achieves full capacity** (unless other resource considerations overrule).

- For 30X30, we would have 900 threads per Block.
  - Only two blocks could fit into an SM for Volta,
  - 1800/2048 ~= 88% of the SM thread capacity would be utilized.

NEW: Thread Block Clusters (TBCs)

- **H100** introduces a new thread block cluster architecture that exposes control of locality at a granularity larger than a single thread block on a single SM.
  - TBCs add another level to the GPU's physical programming hierarchy to include threads, thread blocks, thread block clusters, and grids.
- TBC is a group of thread blocks that are guaranteed to be concurrently scheduled onto a group of SMs, where the goal is to enable efficient cooperation of threads across multiple SMs.
  - The clusters in H100 run concurrently across SMs within a GPC.
- A **GPC** is a group of SMs in the hardware hierarchy that are always physically close together. Clusters have hardware-accelerated barriers and new memory access collaboration capabilities



Nsight Visual Studio Edition (VSE): Integrated GPU Debugger

https://developer.nvidia.com/nsight-visual-studio-edition

- Supports stepping into CUDA kernel code on device
- Powerful capabilities to use integrated Visual Studio debugging with CUDA
  - view warps
  - view GPU registers
  - view per-thread kernel variables
  - more
- Also: can use printf() from kernels!
  - Do not use to troubleshoot race/deadlock conditions or profiling
  - Remove in Release builds
- NVCC kernel resource usage output
  - Use `--resource-usage` compiler flag
  - In VS22: Project Properties/Configuration Properties/CUDA C/C++ / Command Line / Additional Options

- LIVE DEMO

Homework #1 (HW1)

Autumn 2026 - GPU Computing

DUE: 20 October 2026, 5:59 PM

Learning Objectives

- use the CUDA error checking macro (see L1 slides!)
- implement kernels which use multidimensional grids and indexing
- manage host and device explicit memory allocation and data transfer
- launch GPU device kernels from host using multidimensional execution configurations
- practice row-major access (index arithmetic) of 2D / 3D flattened/linearized datasets
- implement range boundary-checking inside kernels
- Perform result verification in the host application, between CPU baseline reference and GPU compute kernel versions, accounting for the possibility of numerical precision issues
- get experience implementing additional simple multidimensional GPU CUDA kernels